DREW

Lecture 14: LC-3 Instructions

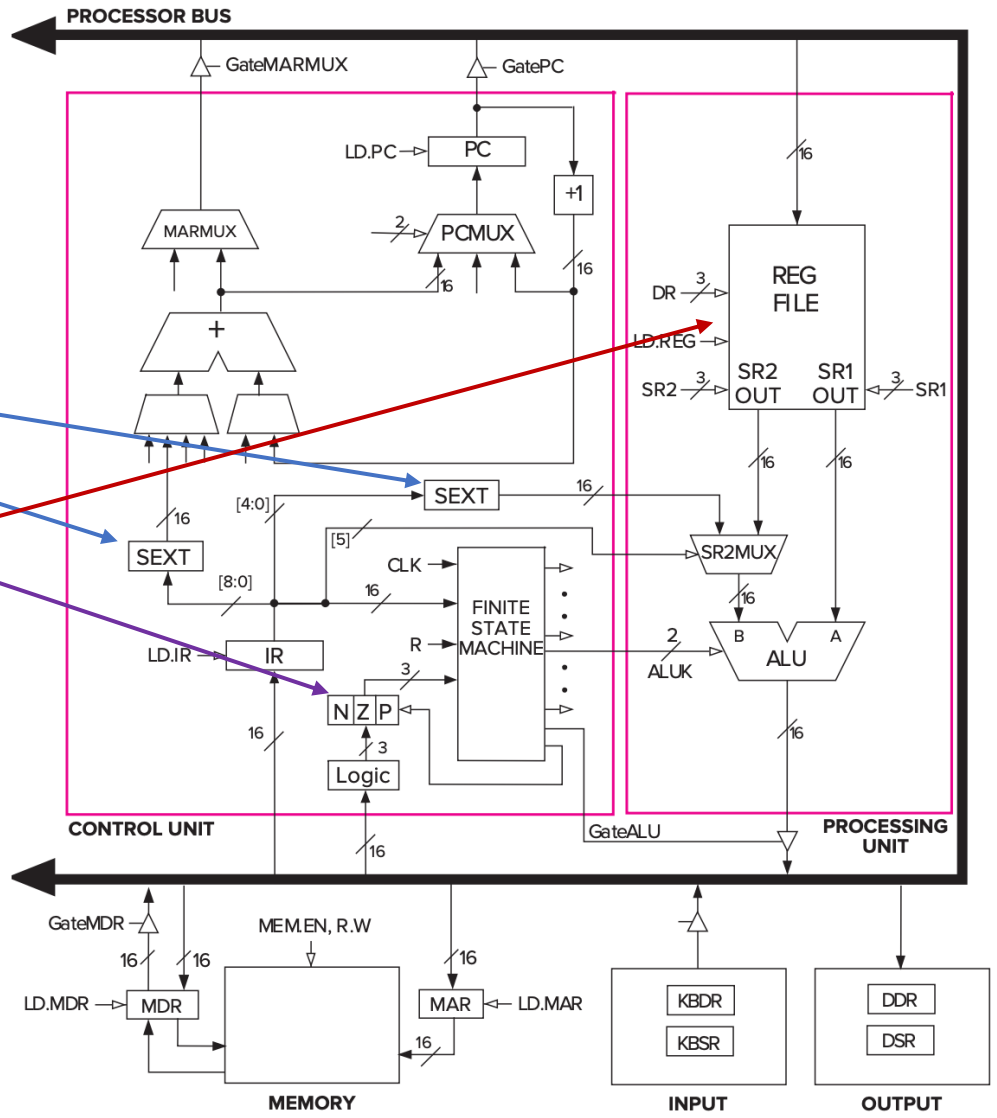
CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

09/30/2024

The Little-Computer 3 Simulator

- Recall the LC-3 data path
  - **SEXT**: Sign-Extended
  - **NZP**: Negative-Zero-Positive
  - **REG FILE**: contains R0~R7 data



The Little-Computer 3 Simulator

- *https://wchargin.com/lc3web/*

- 8 16-bit registers, 2-byte addressable mem.
- Can modify memory/registers with hex sequences.

Status

Registers

R0: x0000	**R1**: x0000	**R2**: x0000	**R3**: x0000
R4: x0000	**R5**: x0000	**R6**: x0000	**R7**: x0000
PC: x3000	**IR**: x0000	**PSR**: x8002	**CC**: Z

Clear R0–R7 | Reset all registers

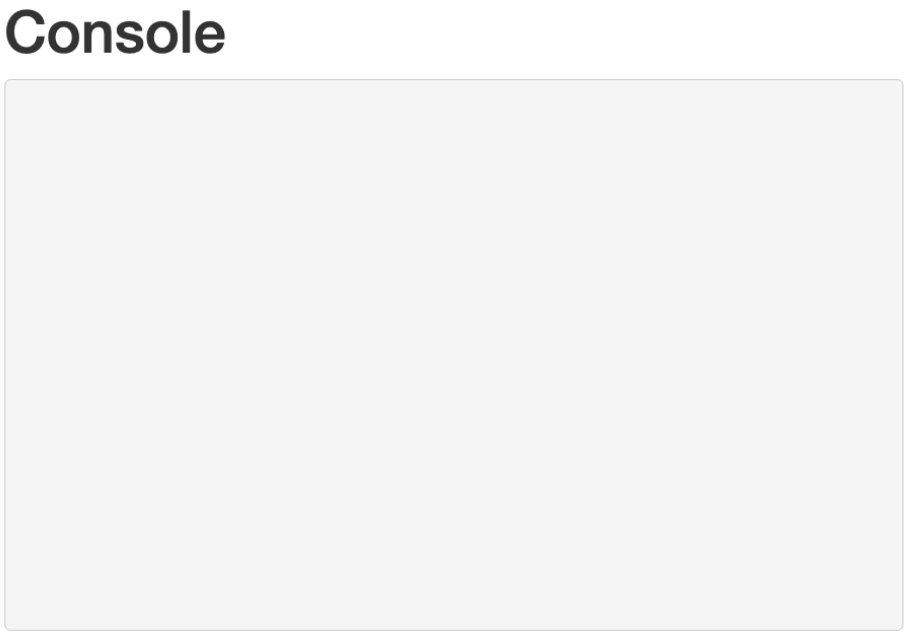
Memory

Jump to address or label | Manage Labels

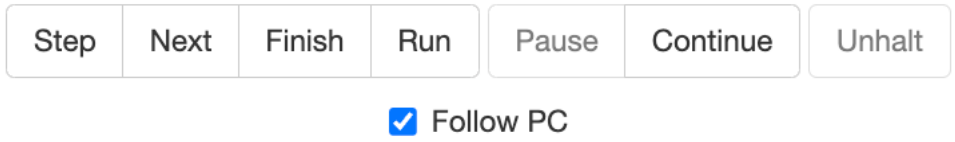
0x	Label	Hex	Instruction
x3000		x0000	NOP
x3001		x0000	NOP
x3002		x0000	NOP
x3003		x0000	NOP
x3004		x0000	NOP
x3005		x0000	NOP
x3006		x0000	NOP
x3007		x0000	NOP
x3008		x0000	NOP
x3009		x0000	NOP
x300A		x0000	NOP
x300B		x0000	NOP
x300C		x0000	NOP
x300D		x0000	NOP
x300E		x0000	NOP
x300F		x0000	NOP

The Little-Computer 3 Simulator

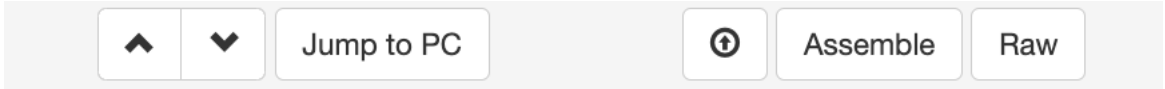
- *https://wchargin.com/lc3web/*



- I/O Console
- Running options: like the debugging options



- Tools: Later on, load machine(bin or hex)/assembly programs as input via Raw/Assemble.



Instructions

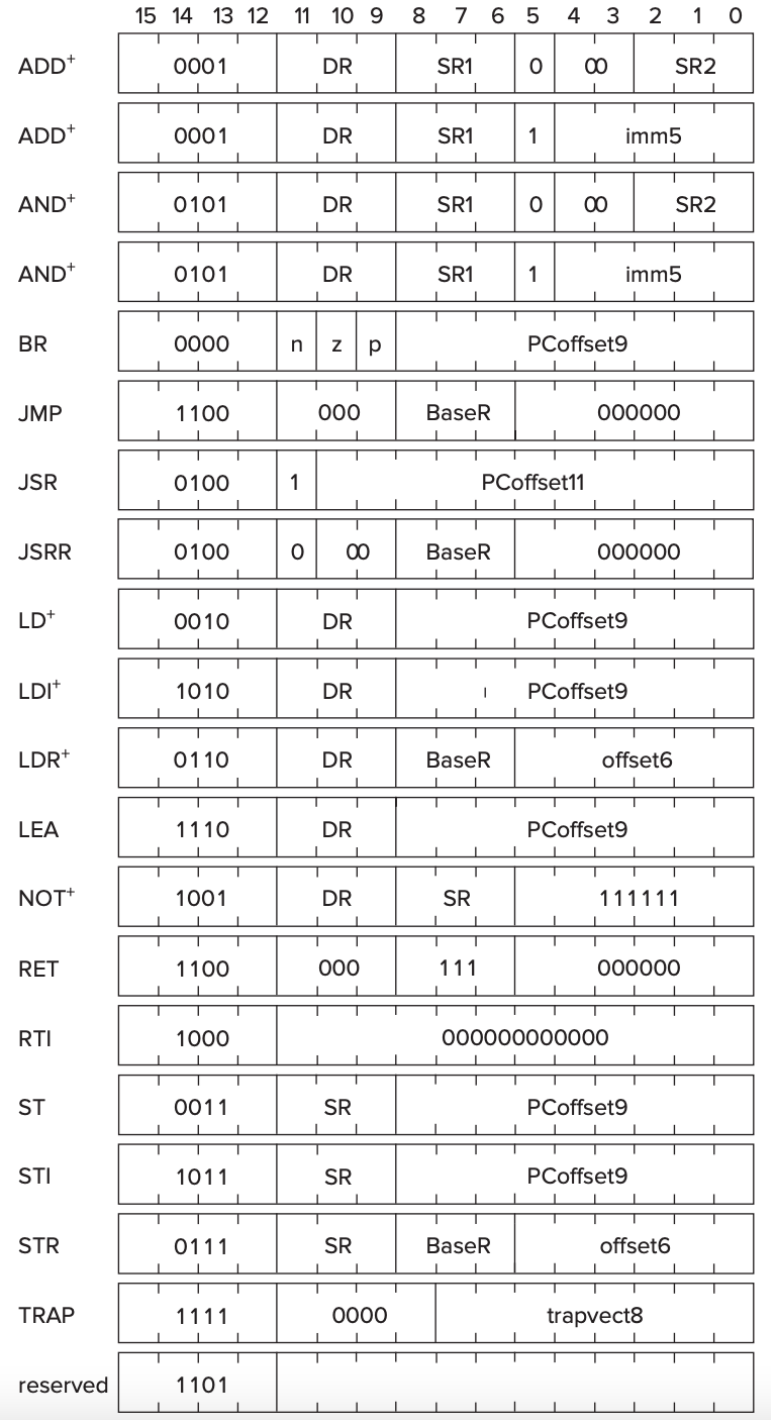
- All the LC-3 instructions:
- 3 Operations, 7 Data Movements, and 3 Controls with variations

- Look at each and everyone of them;

test examples in the simulator.

Note:

- All data in LC-3 are in *2's complement*; *i.e.*, Signed.



DREW

3 Operations

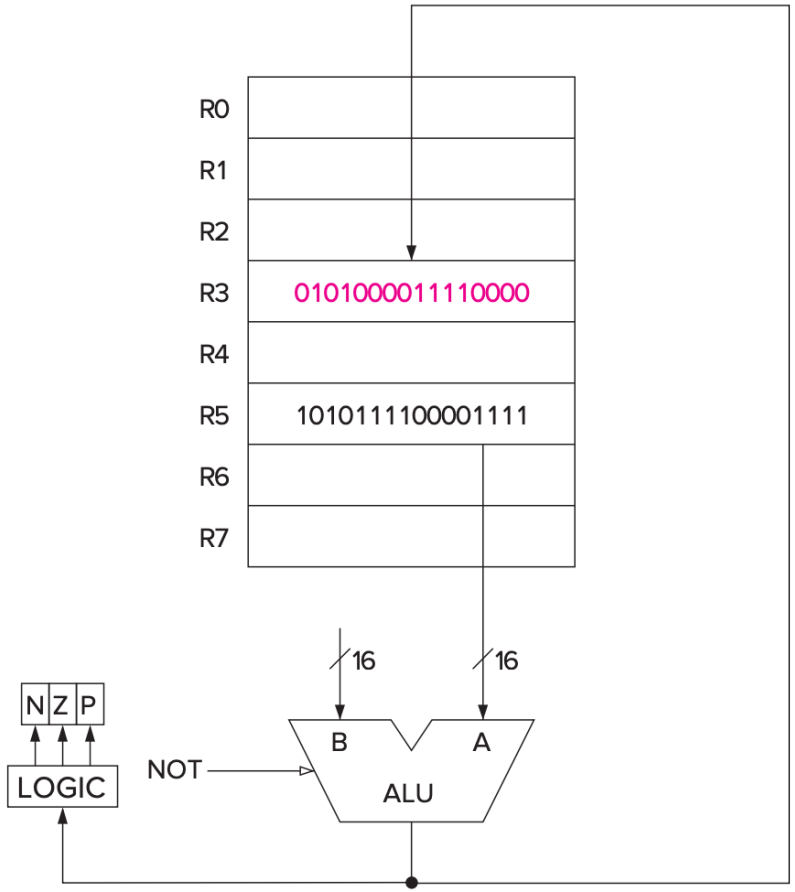
1 Arithmetic
2 Logical
No memory access; so typically done in 1 clock cycle only.

NOT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	DR			SR			1	1	1	1	1	1

- **Opcode**: 1001 **2 operands**: a destination register (R0~R7) and a source register (R0~R7); i.e., 000~111. In which DR and SR can be the same register. **Function**: DR ← **NOT**(SR)
- **Ex.**: 1001 011 101 111111 [NOT R5 R3]

1. ALU retrieves from R5 in TEMP;
2. ALU negates the value;
3. ALU writes result to R3 in TEMP;
4. ALU sets the Condition Code (CC).
   - If result is zero, N=0 Z=1 P=0;
   - If result is positive, N=0 Z=0 P=1;
   - If result is negative, N=1 Z=0 P=0.



DREW

AND Imm5

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR			1	IMM5				

- **Opcode**: 1001 **3 operands**: a destination register (R0~R7), a source register (R0~R7), and a 5-bit number. **Function**: DR ← SR **AND** IMM5

ADD Imm5

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR			1	IMM5				

- **Opcode**: 0001 **3 operands**: a destination register (R0~R7), a source register (R0~R7), and a 5-bit number. **Function**: DR ← SR + IMM5

1. IMM5 is to be SEXT (signed-extended) to 16 bits.
   [Notice the range of IMM5: x-10~xF.]
2. Then send SR and IMM to ALU; store result; again, set the CC.

DREW

AND Register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			0	0	0	SR2		

- **Opcode**: 1001 **3 operands**: a destination register (R0~R7) and two source registers (R0~R7). **Function**: DR ← SR1 **AND** SR2

ADD Register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			0	0	0	SR2		

- **Opcode**: 0001 **3 operands**: a destination register (R0~R7) and two source registers (R0~R7). **Function**: DR ← SR1 + SR2

- Again, will set the CC.

Examples

Set R2 = 1000 1011 1100 0001 (x8BC1), R3 = 0001 0100 1111 1010 (x14FA),
R7 = xFFFF, R0 = 1111000101010111 (xF157), R4 = 0100000010110001 (x40B1).

Address	**Instruction**	**Ins. in Bin**	**Ins. In Hex**
x3000	NOT R1, R1	1001 001 001 111111	x927F
x3001	AND R2,R2,R3	0101 010 010 0 00 011	x5483
x3002	AND R5, R7, 00110	0101 101 111 1 00110	x5BE6
x3003	ADD R6, R4, R0	0001 110 100 0 00 000	x1D00
x3004	ADD R4, R2, 11010	0001 100 010 1 11010	x18BA

7 Data Movements

4 Loads
3 Stores
Loads and Stores with 3 *Addressing Mode*; PC-relative, Indirect, and Base+Offset.

LD PC-relative



- **Opcode**: 0010 **2 operands**: a destination register (R0~R7) and a 9-bit number to add to PC. **Function**: DR ← Mem[PC+PCoffset9]

ST PC-relative



- **Opcode**: 0011 **2 operands**: a source register (R0~R7) and a 9-bit number to add to PC. **Function**: Mem[PC+PCoffset9] ← SR

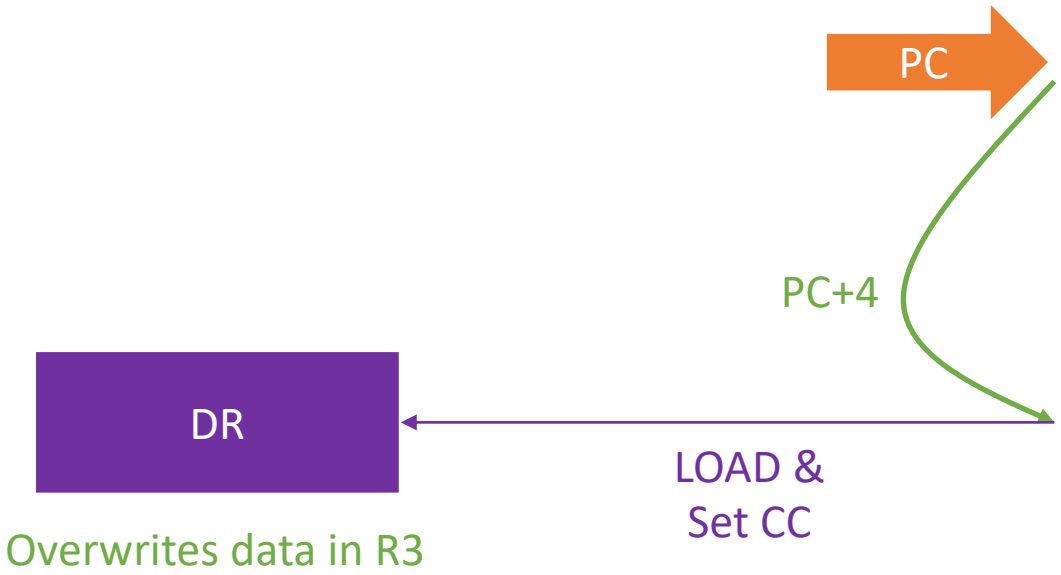
1. SEXT PCoffset9 (#-256~255) to 16 bits and is added to PC.
2. TO LD: Put the sum in MAR. TO ST: Put sum in MAR, data in MDR.
3. TO LD: MDR acquires data from Mem. TO ST: Store in Mem.
4. TO LD: Transfer from MDR to DR and *set CC*.

DREW

LD
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	DR			PCoffset9								

Function: DR ← Mem[PC+PCoffset9]

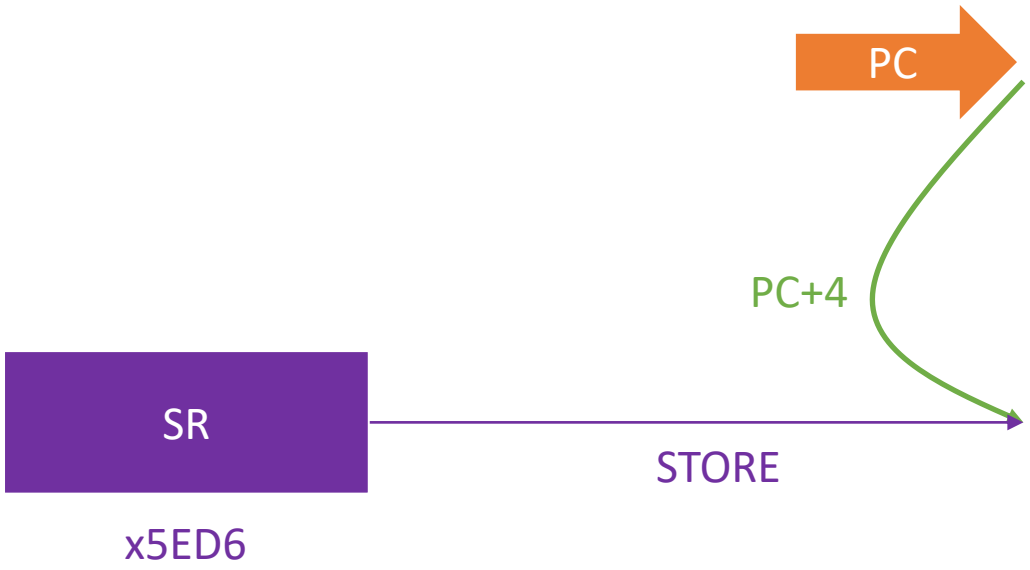


Address	Contents
x3000	… …
x3001	LD R3 #4
x3002	… …
x3003	… …
x3004	… …
x3005	… …
x3006	x114B
x3007	… …

ST
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	SR			PCoffset9								

Function: Mem[PC+PCoffset9] ← SR



Address	Contents
x4100	… …
x4101	ST R3 #4
x4102	… …
x4103	… …
x4104	… …
x4105	… …
x4106	Overwrites data here
x4107	… …

DREW

LD
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	DR			PCoffset9								

ST
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	SR			PCoffset9								

- Again, can only offset PC by #-256~255 places.
- **Q**: What if we want to reach a place that's way far away from the PC?
- **Sol-1**: Use a reachable place to store the address that's far away! And look up the Mem one more time.
- So, LD and ST *indirectly*.

LDI Indirect

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	DR			PCoffset9								

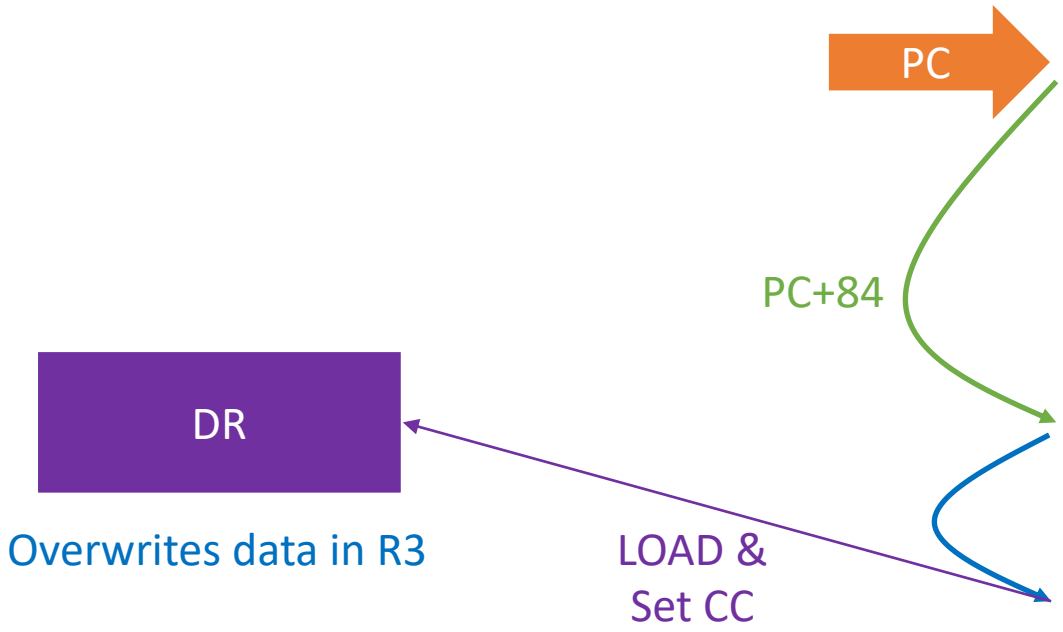
- **Opcode**: 1010 **2 operands**: a destination register (R0~R7) and a 9-bit number to add to PC. **Function**: DR ← Mem[Mem[PC+PCoffset9]]

1. SEXT PCoffset9 (#-256~255) to 16 bits and is added to PC.
2. Put the sum in MAR.
3. MDR acquires data from Mem. (Remember, the data is an address.)
4. MDR puts data in MAR.
5. MDR acquires data from Mem.
6. Transfer from MDR to DR and *set CC*.

LDI Indirect

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	DR			PCoffset9								

Function: DR ← Mem[Mem[PC+PCoffset9]]



Address	Contents
x3000	… …
x3001	LDI R3 #84
x3002	… …
… …	… …
x3054	… …
x3055	… …
x3056	x4100
… …	… …
x4100	xA37C

STI Indirect

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	SR			PCoffset9								

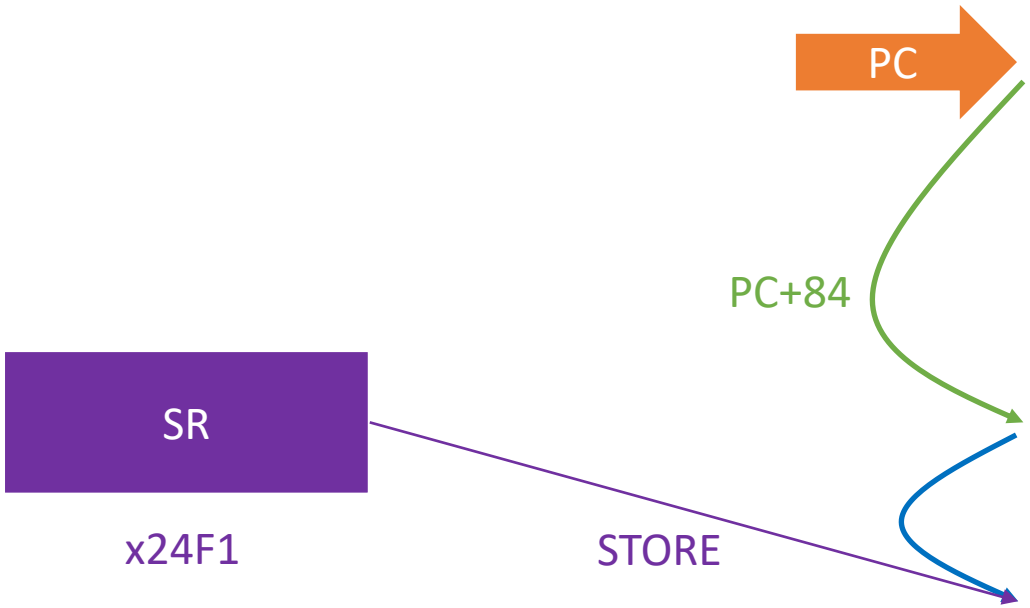
- **Opcode**: 1010 **2 operands**: a source register (R0~R7) and a 9-bit number to add to PC. **Function**: Mem[Mem[PC+PCoffset9]] ← SR

1. SEXT PCoffset9 (#-256~255) to 16 bits and is added to PC.
2. Put the sum in MAR.
3. MDR acquires data from Mem. (Remember, the data is an address.)
4. MDR puts data in MAR.
5. Put data from SR to MDR.
6. Store in Mem.

STI Indirect

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	SR			PCoffset9								

Function: Mem[Mem[PC+PCoffset9]] ← SR



Address	Contents
x3000	… …
x3001	LDI R3 #84
x3002	… …
… …	… …
x3054	… …
x3055	… …
x3056	x4100
… …	… …
x4100	Overwrites data here

LD
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	DR			PCoffset9								

ST
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	SR			PCoffset9								

- Again, can only offset PC by #-256~255 places.
- **Q**: What if we want to reach a place that's way far away from the PC?
  AND, what if there's no space within reachable distance?
- **Sol-2**: Use a register to *generate* the address that's far away! And then look up the Mem.
- So, LD and ST via *register*.

LDR Base+Offset

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	DR			Base			Offset6					

- **Opcode**: 0110 **3 operands**: a destination register (R0~R7), a 3-bit base number (within R0~R7), and a 6-bit number to add to the base.
  Function: DR ← Mem[Reg[Base]+Offset6]

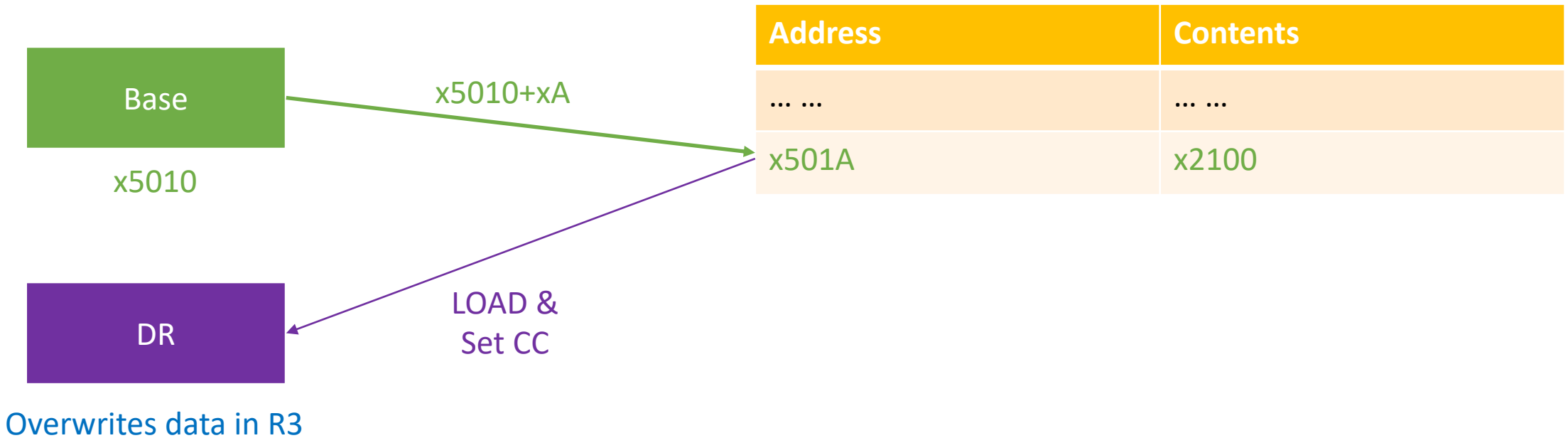
1. SEXT PCoffset6 (#-64~63) to 16 bits and is added to Reg[Base].
2. Put the sum in MAR.
3. MDR acquires data from Mem.
4. Transfer from MDR to DR and *set CC*.

DREW

LDR Base+Offset

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	DR			Base			Offset6					

Function: DR ←
Mem[Reg[Base]+Offset6]



STR Base+Offset

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	SR			Base			Offset6					

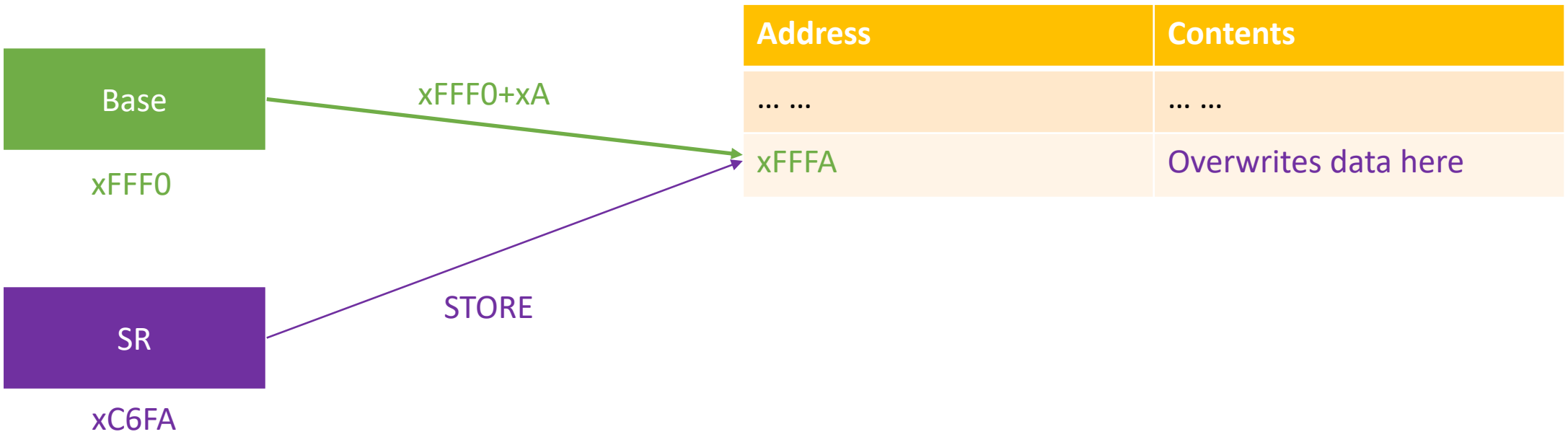
- **Opcode**: 0110 **3 operands**: a source register (R0~R7), a 3-bit indicating R0~R7 whose content is the base, and a 6-bit number to add to the base.
  Function: Mem[Reg[Base]+Offset6] ← SR

1. SEXT PCoffset6 (#-64~63) to 16 bits and is added to Reg[Base].
2. Put the sum in MAR.
3. Put data from SR to MDR.
4. Store in Mem.

STR
Base+Offset

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	SR			Base			Offset6					

Function: Mem[
Reg[Base]+Offset6] ← SR



LEA

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	DR			PCoffset9								

Load Effective Address (Not going through Mem)

- **Opcode**: 1110 **2 operands**: a destination register (R0~R7) and a 9-bit number to add to PC. **Function**: DR ← PC+PCoffset9

1. SEXT PCoffset69 (#-256~255) to 16 bits and is added to PC.
2. Put the sum in DR.

- Note: There's no way to move content from Mem to Mem in one instruction.

Examples

- Set PC = x30F6.

Address	Instruction	Ins. in Bin	Ins. In Hex
x30F6	LEA R1, #-3	1110 001 111111101	xE3FD
x30F7	ADD R2, R1, #14	0001 010 001 1 01110	x146E
x30F8	ST R2, #-5	0011 010 111111011	x35FB
x30F9	AND R2, R2, #0	0101 010 010 1 00000	x54A0
x30FA	ADD R2, R2, #5	0001 010 010 1 00101	x14A5
x30FB	STR R2, R1, #14	0111 010 001 001110	x744E
x30FC	LDI R3, #-9	1010 011 111110111	xA7F7

DREW

3 Control Instructions

Branchings
Jumps
TRAPs

Branch
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	N	Z	P	PCoffset9								

- **Opcode**: 0000 **4 operands**: three single-bit CC checker and a 9-bit PC offset. **Function**: Upon match with current CC, go to location.

- **Recall**: NOT, AND, ADD, LD, LDI, LDR all *sets CC*.
- Based on the result, exactly one flag is set to 1.

- BR uses CC checker to match with the current CC.
  - BRn, BRnz, BRnp, BRnzp, BRz, BRzp, BRp
- If matched, go to PC+PCoffset9 location.

- **Ex.**: if-statement, looping-condition, subroutine calls, etc.

Jump Register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	Base			0	0	0	0	0	0

- **Opcode**: 1100 **1 operand**: a 3-bit indicating R0~R7 whose content is the base. **Function**: Unconditionally branch to location.

1. Assign content in the base register to PC.

TRAP

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	TRAPvect8							

- **Opcode**: 1111 **1 operand**: an 8-bit vector for specifying the TRAP type. **Function**: Calls a *Service Routine* and return to the PC location.
- TRAPvect8: x23: Input a char. from keyboard.
  x21: Output a char. to the monitor.
  x25: Halt the program.

Questions?

- **Short Summary**
  - All the instructions with the 0's,1's are part of the *Machine Language*, what the computer really reads.
  - All programming languages, are ultimately translated to machine language before execution.